Imperial College
London

Reinforcement Learning Coursework 2 - Bioengineering

CID:01907399

Imperial College London

October 2023

1 Question 1: Tuning the DQN

The CartPole environment consists of a pole on a moving cart, which can be pushed to either left or right to keep the pole balanced on the cart for as many steps as possible. A DQN agent was trained for this task, aiming for the episode length to be over 100 for at least 50 consecutive episodes. The mean episode length was taken over 10 runs.

1.1 Hyperparameters

The hyperparameters were chosen through an iterative process described below. The training details and architecture presented in in Mnih et al. [2015] constituted the starting point of the hyperparameter exploration. Starting values were inspired from the paper, with some adaptation to suit the states in the CartPole environment. The hyperparameters were initially adjusted based on their theoretical expected effect until a learning curve that proved to train within the 300 episodes was obtained. Then each hyperparamater was varied while keeping all the others in the set constant. The investigated effects were the converging time, the curve fluctuations, the final mean return, and the variance across the 10 runs. This search allows making informed decisions about how to change hyperparameters to improve results, however it is important to note that the effect of one hyperparameter should not be looked at completely isolated from the others. Choosing the best performing hyperparameter from each of these sweeps might not lead to the best performing combination. Therefore, grid search was performed to find the optimal hyperparameter set. It is also important to take into account the effect of random weight initialisation when the DQN is created. To eliminate that from the hyperparameter comparison, a random seed needs to be implemented.
The hyperparameters explored, the chosen values, and the reasoning behind can be found in Table 2. Ranges explored are presented in Table 1.

Hyperparameter	**Investigated values**
Learning Rate	0.0001, 0.0002, 0.0005, 0.001, 0.002, 0.005, 0.01, 0.02
Adam Betas	[0.9,0.99] ,[0.95,0.95] ,[0.9,0.999] ,[0.99,0.99]
Batch Size	1, 2, 4, 8, 16, 32, 64
Optimisation Frequency	1, 2, 4, 8, 10
Discount Factor	1, 0.99, 0.95, 0.9
Initial Exploration and decay rate	[1,linear-0.99], [1,exp-0.99],[0.9,linear-0.99] [0.9,exp-0.99],[0.8,linear-0.99],[0.7,linear-0.99]
Minimum Exploration	0.1
Target Update Frequency	[1, 2, 4, 5, 8, 10, 16, 20]
Replay Memory Buffer	100, 1000, 10000, 20000, 50000, 100000
Replay Start Size	batch size, 2× batch size, 10 × batch size, 1000
Architecture	[4,3,2], [4,3,3,3,2], [4,4,4,4,2], [4,64,2], [4,128,2], [4,64,64,2], [4,128,128,2], [4,64,64,64,2], [4,128,128,128,2]
Activation Function	LeakyReLU, ReLU

Table 1: Investigated values of hyperparameters

Hyperparameter	**Value**	**Justification**
Optimizer choice	Adam	Adam (Kingma and Ba [2014]) was chosen for the adaptive learning rate used in stochastic gradient descent, which is based on the statistics of the previous gradients. Commonly used in deep learning.
Learning Rate (α)	0.001	Adam hyperparameter, it determines the step size used in gradient descent. Empirical results confirmed the theory that large values result in high fluctuations and overshooting the optimal value, while low values present less variability but slow training. 0.0005, had slightly higher return than 0.001, but converged slower, and in in combination with other hyperparameters, convergence was too slow such that over 300 episodes the return was higher with with 0.001.
Moment estimates β_1, β_2	0.9,0.99	Adam hyperparameters, they control the exponential decay rates by setting the amount of information that is retained about previous gradients. Empirical results showed that higher β_1 resulted in smoother estimates but slowed down training. For β_2, 0.999 resulted in a more stable learning curve, however 0.99 resulted in a higher mean return.
Batch Size	32	Empirical results confirmed theory as low values resulted in high fluctuations, while higher values increased runtime. 32 and 64 had similar performance, so 32 was chosen to reduce computational effort. Powers of 2 were tested to take advantage of the GPU architecture.
Optimisation Frequency	4	Sets the number of steps after which gradient descent is performed to update the parameters in the policy network. Higher frequency resulted in high fluctuations, while lower frequencies resulted in slow convergence. Chosen value showed the best trade-off between the 2 and highest mean return in the last 100 episodes.
Discount Factor (γ)	1	Controls the importance of future rewards. Considering the goal of our environment, all future rewards should be taken into account, so a value of 1 was chosen. Empirically, varying (γ) did not show significant changes.
Initial Exploration ϵ	1.0	Establishes the agent's exploration rate. Starting with a high value ensures more exploration in the beginning of training. While this created higher fluctuations in the learning curve, it was proven useful as convergence value was higher than when starting with less exploration.

Minimum Exploration	0.1	A minimum ϵ value was set to ensure the agent is still exploring at late stages of training. Value was taken from Mnih et al. [2015]
ϵ Decay Date	0.99	Different linear and exponential decay rates were tested together with different initial exploration values. The chosen combination presented slightly higher variability but highest mean return and fastest convergence.
Target Update Frequency	4	Number of steps after which the target network is updated. Increased values reduce the correlation between estimated Q-values and target values and therefore stabilize training. The value was chosen as it gave the highest return among the converging learning curves obtained.
Replay Memory Buffer	20000	Size of the replay memory buffer. Increasing the size resulted in higher final return, as the agent learns from a more diverse set of experiences. The return did not increase significantly past 20000 states so the value was chosen to limit the memory requirements without affecting the return.
Replay Start Size	batch size	Sets the minimum number of steps stored in the buffer before the agent starts starts sampling experiences. Minimum possible number is equal to the batch size, however increasing it can create more variety and less correlations among the states being sampled at different steps. However empirical results did not show significant improvement.
Hidden Layers Tyoe	fully-connected	Linear layers were chosen as they are suitable for our 1-D input data.
Architecture	[4,64,64,2]	This architecture for its fast and stable training. Wider networks resulted in faster training but lower return, while deeper networks resulted in faster but unstable training, with high fluctuations. Small hidden layer sizes of 3 to 16 proved to not capture the complexity of the data and resulted in the agent not training, even for higher depth.
Activation Function	LeakyReLU	Mitigates risk of dying ReLU caused by negative gradients being set to 0. It is less computationally expensive than other activation functions like Mish, Sigmoid, Tanh, which are also not expected to be suitable for this input data.
Loss Function	Mean squared error	Standard loss function for regression tasks. Q-value approximation can be seen as a regression problem.

Table 2: Hyperparameters chosen values for CartPole DQN

1.2 Learning curve

Figure 1 shows the learning curve obtained using the hyperparameters presented in Table 1. Learning begins after a few more than 5o episodes, and the return crosses the 100 threshold after around 150 episodes. The learning curve converges after 200 episodes at a return of approximately 270, calculated as the mean over 100 episodes. The combination of parameters was chosen for the stable training and high convergence value. However, lower variance would be desired.

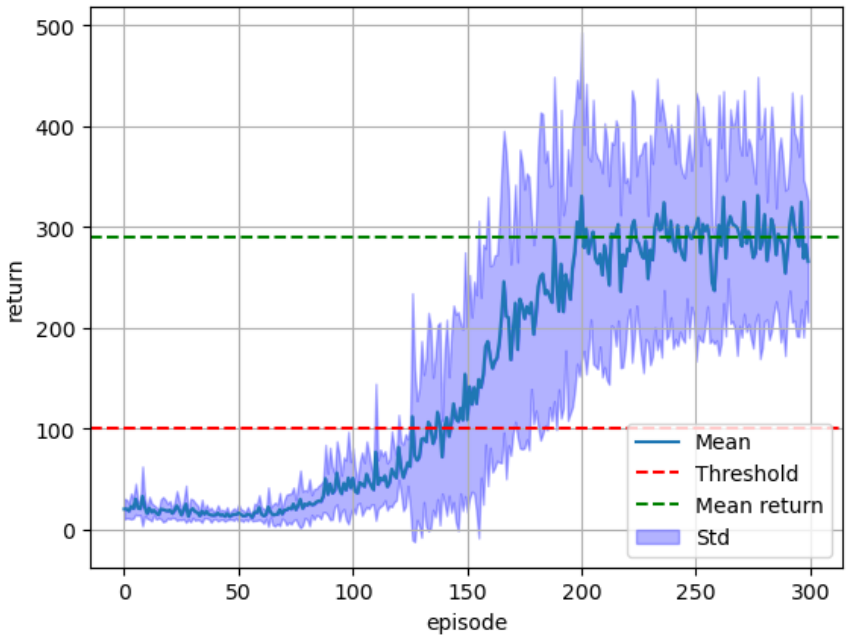


Figure 1: Learning curve of DDQN. Mean return is the mean over the last 100 episodes.

2 Visualisation of DQN policy

We can visualise in 2D the policy and Q-value functions associated with states as a function of 2 out of 4 of the variables that define the state, by setting the other 2 constant. The policy and Q-values were investigates for states in which the cart is at position 0, for velocity values of 0, 0.5, 1, and 2, as a function of pole angle and pole angular velocity.

2.1 Slices of the greedy policy action

Figure 2 shows how the policy changes as the speed of the cart towards right increases. It can be seen how, when the cart is not moving, if the pole is tilted towards the right, i.e. angle is positive, for positive angular velocities, the best action is to push the cart to the right in order to stabilize the pole. For negative angular velocities, i.e. the pole is rotating back towards the centre, the best action is to push the cart to the left, which will reduce the angular velocity. When the pole is titled to the left, the opposite holds. Increasing the velocity induces a constant negative angular velocity on the pole. What happens to the policy graph can be explained mathematically: We can visualise the boundary between action 0 and action 1 as a linear function, such that $\omega = f(\theta)$. In this case, we have a linear function $\omega = -m \times \theta + n$, with $m > 0$. When an angular velocity ω_0 is induced by the movement of the cart, our plot becomes $\omega = -m \times \theta + n + \omega_0$. For positive velocities, ω_0

is negative, and as velocity increases, it becomes more negative shifting our boundary downwards. Intuitively, as velocity increases, the angular velocity and angle have to be more negative before the agent pushes the cart to the left. This could be explained by the fact that a push to the left will have a higher inertial effect than when stationary, so the agent waits for the angle or angular velocity to become more negative before pushing the cart to the left, to ensure the the upright position is not overshot. Otherwise the agent will choose to push the cart to the right to keep it moving in the same direction. It has to be noted that the negative slope is not perfectly smooth.

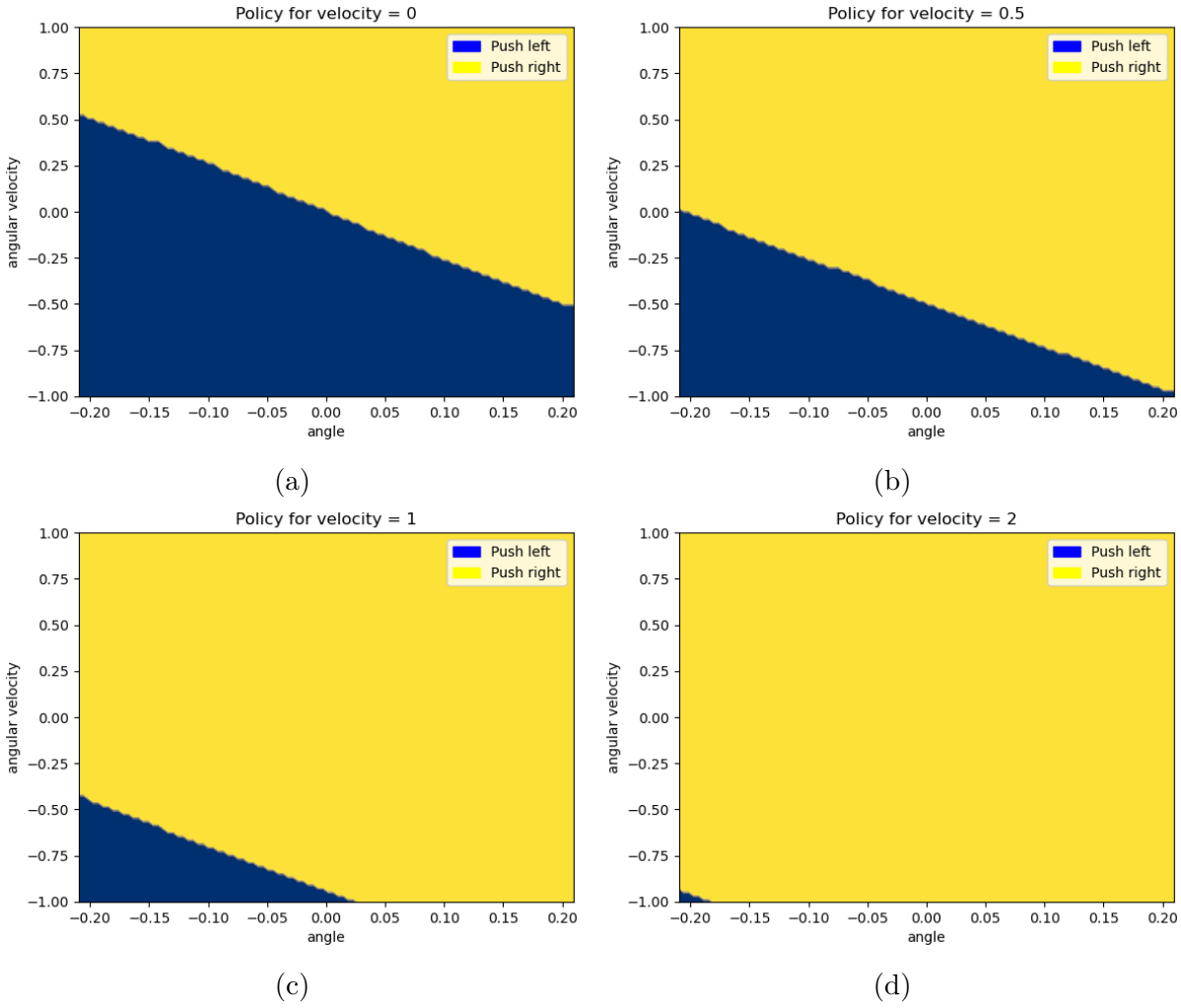
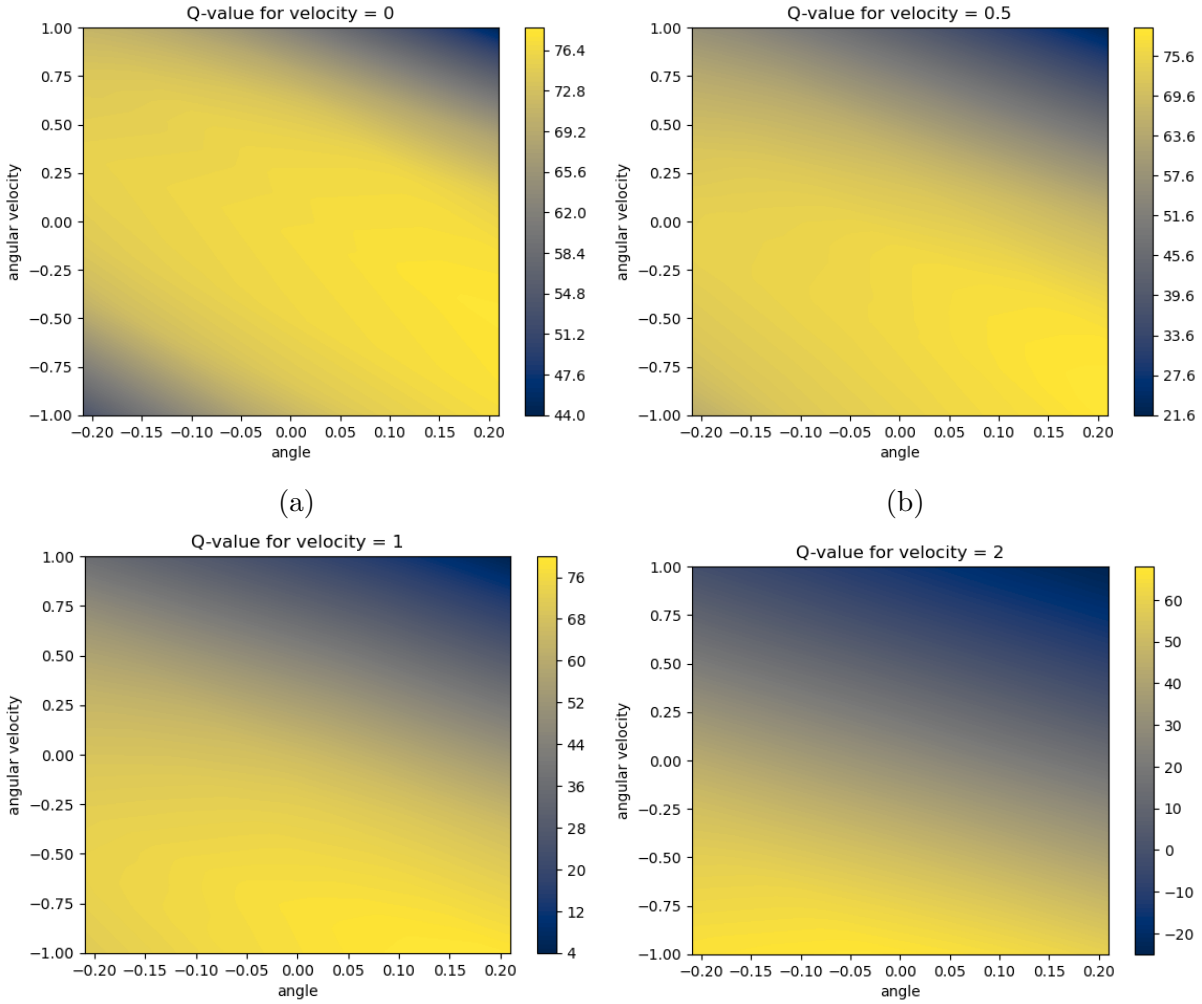


Figure 2: Slices of the policy for cart position 0 and cart velocities of 0, 0.5, 1, 2.

2.2 Slices of the Q function

The Q-value plots help visualize what states are favorable to be in and which ones are not. Figure 3 shows that when the cart is not moving, the most favorable actions are the one in which the pole is rotating towards the centre, with an angular velocity that decreases as the angle decreases. States in which the pole is falling with high angular velocity to either right or wrong, meaning high, same-sign, values for angular velocity and angle, are associated with a low Q-value function, close to 0, as the pole is close to falling out of the range resulting in the episode terminating. As the velocity to the right increases, states with large negative angular velocity and angle have better

chances to be balanced through a push to the left than when the cart is not moving, because of a higher inertial effect. At the same time, states with large positive angular velocity and angle become less favorable, as pushing the cart to the right won't have the same balancing effect as before. It should be noted that overall, the minimum Q-value the states can have as the velocity increases decreases.



(a) (b)

(c) (d)

Figure 3: Slices of the Q function policy for cart position 0 and cart velocities of 0, 0.5, 1, 2.

References

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.